11주차 강의

# Visual Anomaly Detection

김영진
you359@cbnu.ac.kr
충북대학교 산업인공지능연구센터

INDUSTRIAL AI RESEARCH CENTER

# CONTENTS


I **Opensource for VAD**
What is Anomalib, Concepts and Architecture, Core Components

II **VAD Project**
문제·요구사항 → 데이터 → 입력 → 방법 → 운영 규칙 → 시스템 → 평가

## Anomalib

- Github Link : https://github.com/open-edge-platform/anomalib?tab=readme-ov-file
- Documentation Link : https://anomalib.readthedocs.io/en/latest/index.html

README | Code of conduct | Contributing | Apache-2.0 license | Security

# ANOMALiB

A library for benchmarking, developing and deploying deep learning anomaly detection algorithms

Key Features • Docs • Notebooks • License

python 3.10+ | pytorch 2.0+ | lightning 2.2+ | openvino 2024.0+

Pre-Merge Checks failing | codecov 80% | PyPI Downloads 1M | package health 84/100 | openssf best practices silver

docs passing | Gurubase Ask Anomalib Guru

6 GITHUB TRENDING
#6 Repository Of The Day

## What is Anomalib

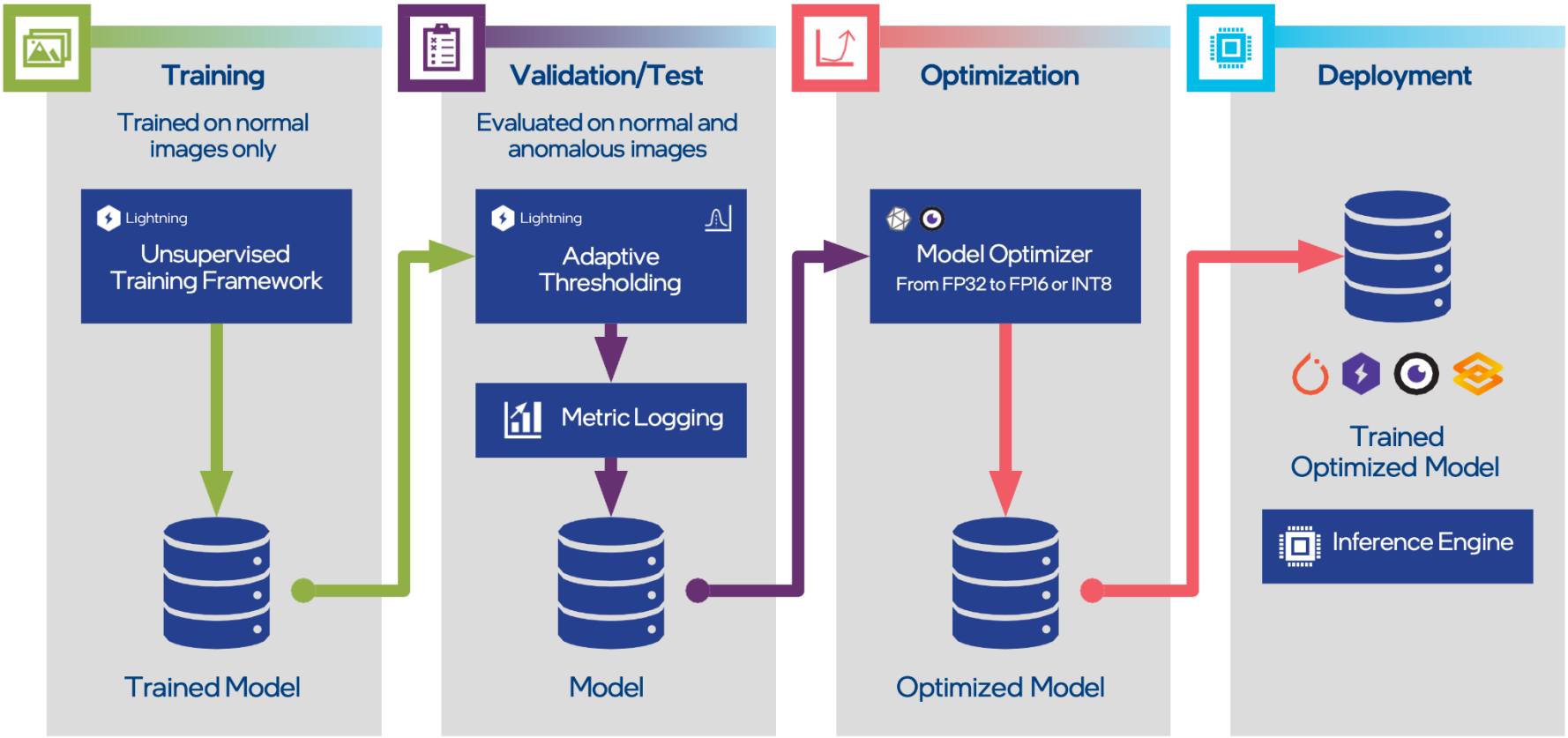
- Anomalib는 공개 및 비공개 데이터셋 모두에서 **최신 이상 탐지 알고리즘을 수집하고 벤치마킹**하기 위한 딥러닝 라이브러리
- 이 라이브러리는 **최근 연구에서 제안된 여러 이상 탐지 알고리즘의 즉시 사용 가능한 구현체를 제공**하며, 사용자가 **커스텀 모델을 손쉽게 개발하고 적용할 수 있도록 돕는 다양한 도구 세트를 포함**
- Anomalib의 주요 초점은 **시각적 이상 탐지(Visual Anomaly Detection)**,
  데이터셋 내의 이미지나 비디오에서 이상 징후를 탐지하거나 위치를 찾아내는 것을 목표로 합니다.

## Anomalib 주요 특징

- Modular Design
- Wide Algorithm Support
- Real-time Detection
- Visualization Tools
- Customization and Extensibility
- Comprehensive Documentation and Community Support

## Concept of Anomalib

- High-Level Workflow



## Concept of Anomalib

- High-Level Workflow

- **Training [Optional]**: Involves **training** the anomaly detection model to learn patterns and characteristics of normal behavior. This step is optional for unsupervised algorithms. *For zero-shot learning, the model is not trained on any specific dataset*.

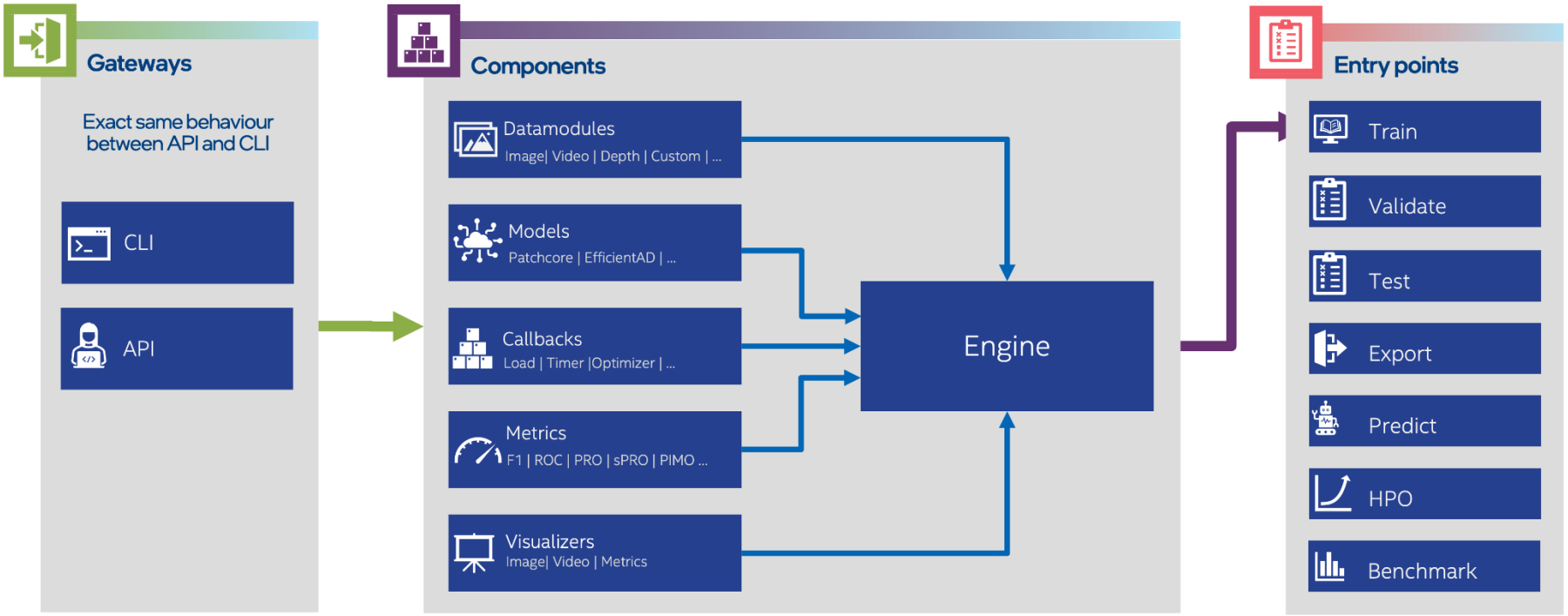
- **Validation/Test**: **Validates** the model by **finding the threshold** for anomaly detection and **testing** the model's performance on a validation or test dataset.

- **Optimization [Optional]**: Involves optimizing the model by converting it to a more efficient form, such as a **compressed or quantized model**. This step utilizes tools such as ONNX, OpenVINO for optimization.

- **Deployment**: **Deploys** the model to a production environment for real-time or batch anomaly detection. This step uses multiple tools such as Torch, Lightning, ONNX, OpenVINO and Gradio for deployment.

## Anomalib Architecture

- High-Level Architecture



## Anomalib Architecture

- High-Level Architecture

- **API/CLI**: Provides an **interface** for users to **interact with the library**, including APIs for data ingestion, configuration, and result export, as well as a command-line interface for running anomaly detection tasks. Both API and CLI provides the same functionality.

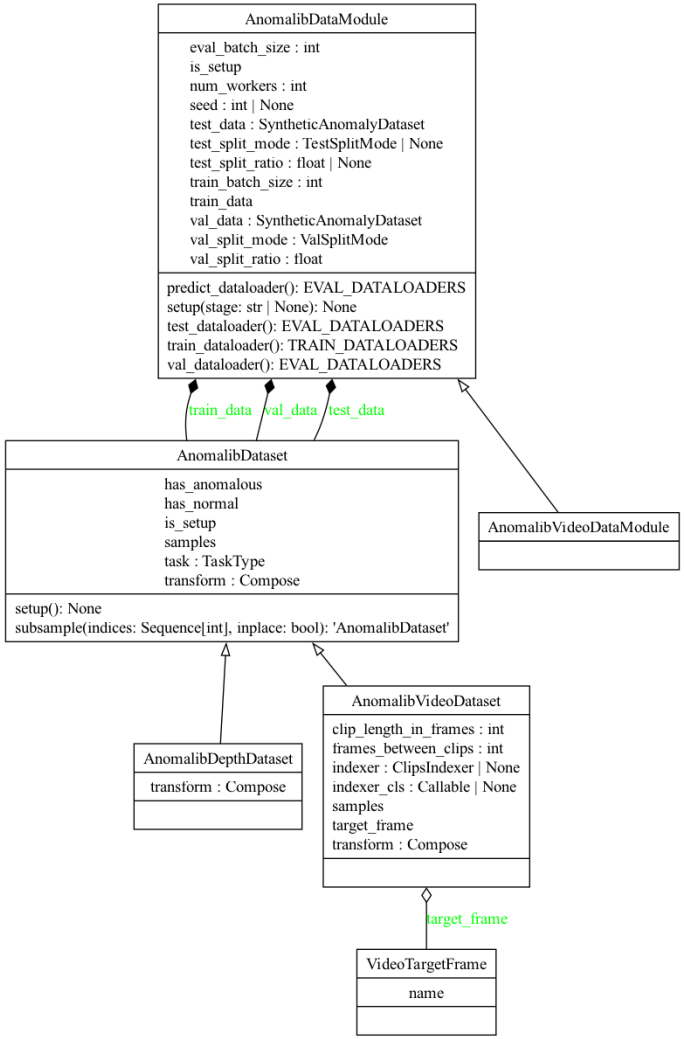
- **Components**: Consists of the **core components** of Anomalib, including, **Datamodules, Models, Callbacks, Metrics, Visualizers and Engine**.

- **Entry Points**: Includes the **main entry points** for running anomaly detection tasks, such as **train, validate, test, export, and predict**

# Anomalib Core Components

## Core Components 1: Datamodules

- Datamodules in anomalib are responsible for **preparing the data** for training, validation, and testing.
- **loading, preprocessing, and splitting** the data
- sub-classes by image, video and depth datamodules



# Anomalib Core Components

INDUSTRIAL AI RESEARCH CENTER

## Core Components 2: Models

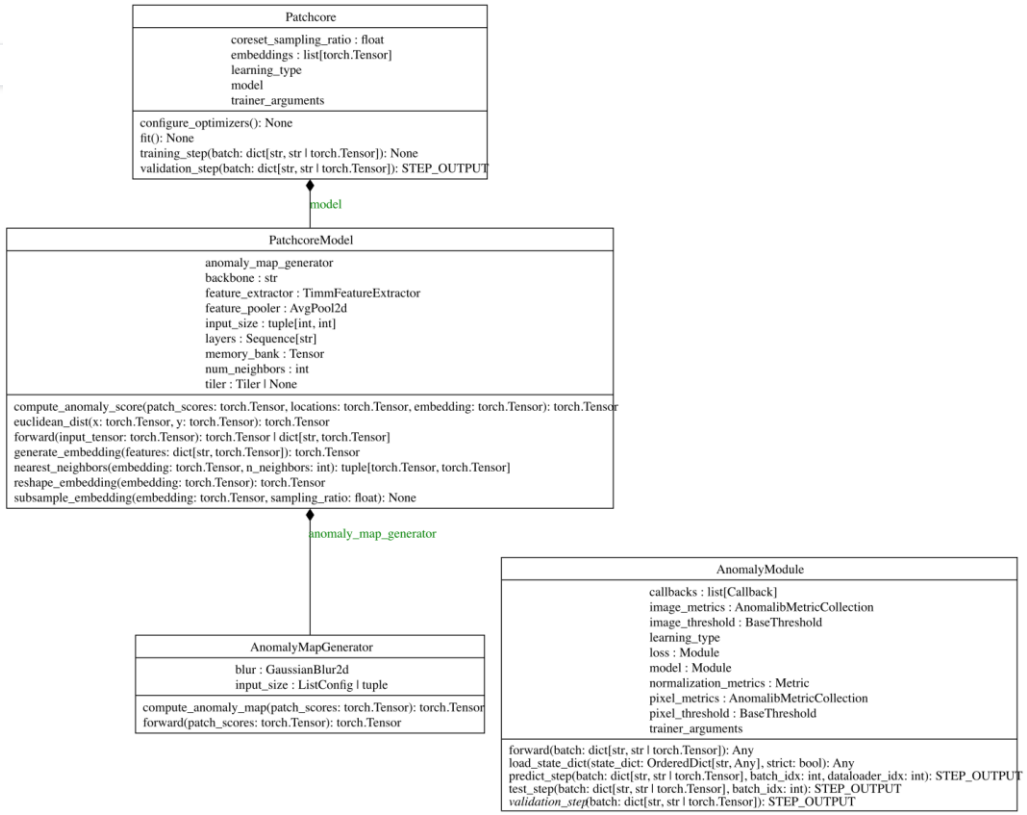
- Anomalib provides a **collection of anomaly models** within the image and video domains.

### Image Models

- **CFA** — Coupled-hypersphere-based Feature Adaptation for Target-Oriented Anomaly Localization
- **C-Flow** — Real-Time Unsupervised Anomaly Detection via Conditional Normalizing Flows
- **CS-Flow** — Fully Convolutional Cross-Scale-Flows for Image-based Defect Detection
- **DFKDE** — Deep Feature Kernel Density Estimation
- **DFM** — Probabilistic Modeling of Deep Features for Out-of-Distribution and Adversarial Detection
- **DRAEM** — DRÆM – A discriminatively trained reconstruction embedding for surface anomaly detection
- **DSR** — DSR – A Dual Subspace Re-Projection Network for Surface Anomaly Detection
- **Efficient AD** — EfficientAD: Accurate Visual Anomaly Detection at Millisecond-Level Latencies
- **FastFlow** — FastFlow: Unsupervised Anomaly Detection and Localization via 2D Normalizing Flows
- **FRE** — FRE: A Fast Method For Anomaly Detection And Segmentation
- **GANomaly** — GANomaly: Semi-Supervised Anomaly Detection via Adversarial Training
- **PaDiM** — PaDiM: A Patch Distribution Modeling Framework for Anomaly Detection and Localization
- **Patchcore** — Towards Total Recall in Industrial Anomaly Detection
- **Reverse Distillation** — Anomaly Detection via Reverse Distillation from One-Class Embedding.
- **STFPM** — Student-Teacher Feature Pyramid Matching for Unsupervised Anomaly Detection
- **SuperSimpleNet** — SuperSimpleNet: A Unified Surface Defect Detection Model for all Supervision Regimes
- **U-Flow** — U-Flow: A U-shaped Normalizing Flow for Anomaly Detection with Unsupervised Threshold
- **VLM-AD** — VLM-AD: Vision-Language Model for Anomaly Detection
- **WinCLIP** — WinCLIP: Zero-/Few-Shot Anomaly Classification and Segmentation

### Video Models

- **AI VAD**



# Anomalib Core Components



## Core Components 3: Callbacks

- We use LightningCallbacks to add functionality to the training loop, such as **logging, early stopping, and model checkpointing**
- Lightning Callbacks Documentation : https://lightning.ai/docs/pytorch/stable/extensions/callbacks.html

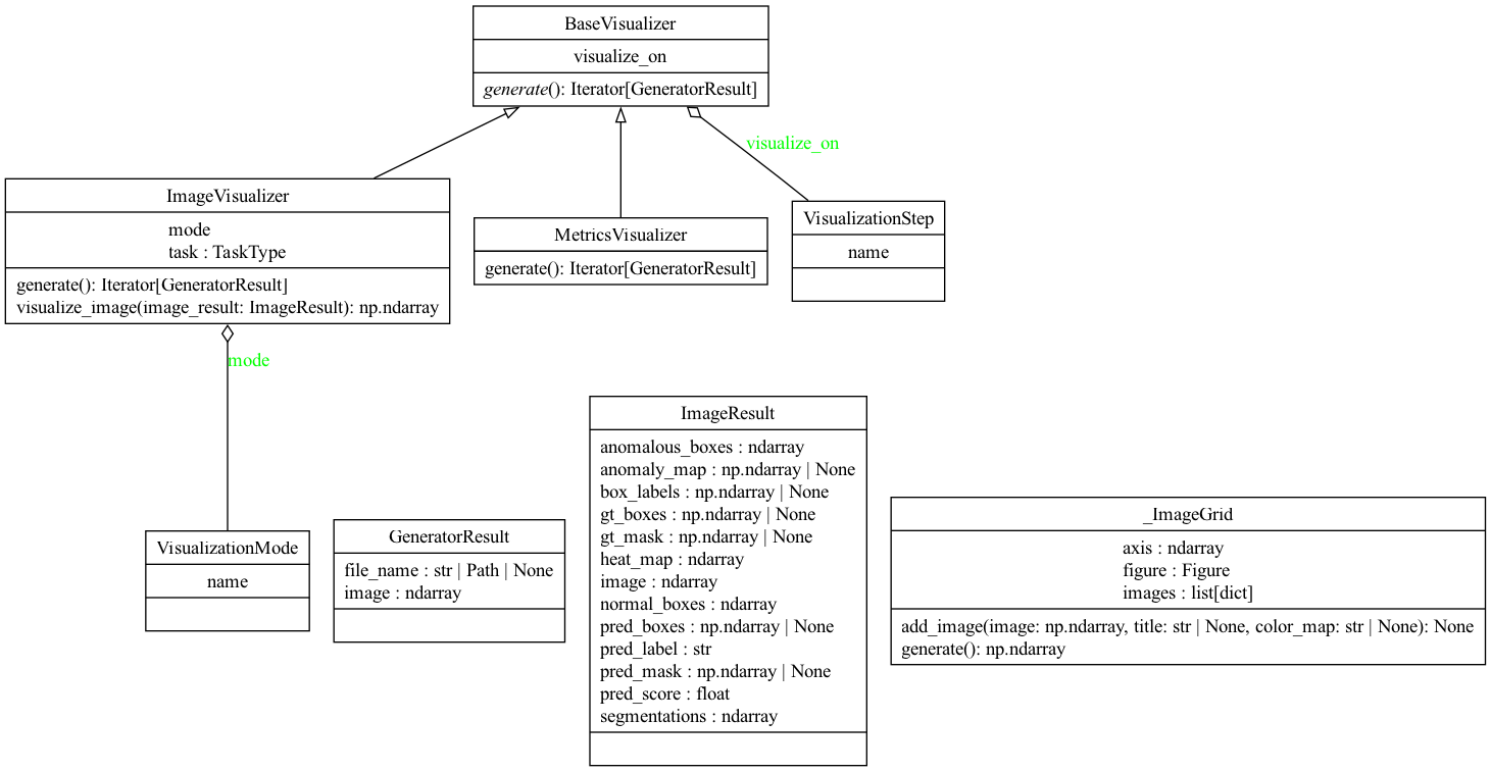
## Core Components 4: Metrics

- Similar to callbacks, we make use of Lightning Torchmetrics package, which provides a collection of metrics **for evaluating model performance**.
- We have also implemented custom metrics such as PRO, sPRO and PIMO for anomaly detection tasks, which could be accessed through the metrics module.
- Torchmetrics documentation : https://lightning.ai/docs/torchmetrics/stable/

# Anomalib Core Components

## Core Components 5: Visualizers

- Anomalib provides a suite of **visualization tools** for analyzing anomalies, understanding model decisions, and interpreting results.

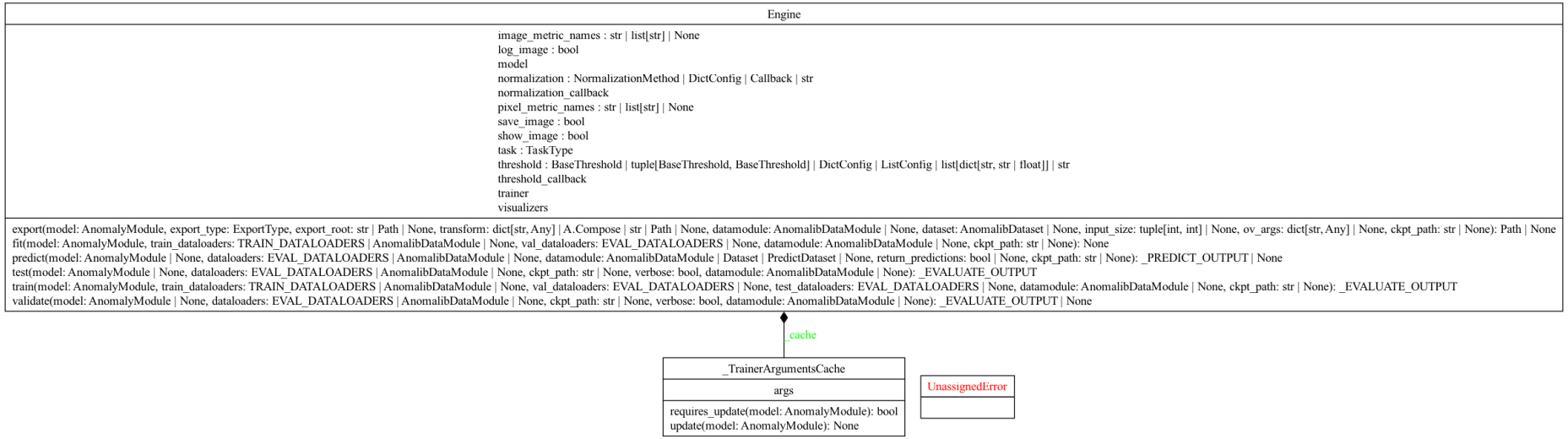


# Anomalib Core Components

INDUSTRIAL AI RESEARCH CENTER

## Core Components 6: Engine

- The figure below demonstrates the high-level architecture of the engine, which is responsible for **orchestrating the anomaly detection workflow**, including data preparation, model training, validation, testing, export and predict.



# Anomalib Experiments

## Installation

### Default Installation

```
# With uv
uv pip install anomalib

# Or with pip
pip install anomalib
```

### Advanced Installation

```
# To ensure a specific hardware backend, you must specify an extra.
# For example, for CPU:
pip install "anomalib[cpu]"

# Or for CUDA 12.4:
pip install "anomalib[cu124]"

# For a full installation with all optional dependencies on CPU:
pip install "anomalib[full,cpu]"
```

INDUSTRIAL AI RESEARCH CENTER

## Training

### Training with API or CLI

API | CLI

```python
# 1. Import required modules
from anomalib.data import MVTecAD
from anomalib.deploy import ExportType
from anomalib.engine import Engine
from anomalib.models import Patchcore

# 2. Create a dataset
# MVTecAD is a popular dataset for anomaly detection
datamodule = MVTecAD(
    root="./datasets/MVTecAD",  # Path to download/store the dataset
    category="bottle",  # MVTec category to use
    train_batch_size=32,  # Number of images per training batch
    eval_batch_size=32,  # Number of images per validation/test batch
)

# 3. Initialize the model
# Patchcore is a good choice for beginners
model = Patchcore(
    num_neighbors=6,  # Override default model settings
)

# 4. Create the training engine
engine = Engine(
    max_epochs=1,  # Override default trainer settings
)

# 5. Train the model
# This produces a lightning model (.ckpt)
engine.fit(datamodule=datamodule, model=model)

# 6. Test the model performance
test_results = engine.test(datamodule=datamodule, model=model)

# 7. Export the model
# Different formats are available: Torch, OpenVINO, ONNX
engine.export(
    model=model,
    export_type=ExportType.OPENVINO,
)
```

API | CLI

```bash
# 1. Basic Training
# Train and test a model using default configuration on MVTecAD bottle (default category)
echo "Training with default configuration..."
anomalib train --model Patchcore --data anomalib.data.MVTecAD

# 2. Training with Basic Customization
# Customize basic parameters like batch size and epochs.
# For example `EfficientAd` requires a train batch size of 1
echo -e "\nTraining with custom parameters..."
anomalib train --model EfficientAd --data anomalib.data.MVTecAD \
    --data.category hazelnut \
    --data.train_batch_size 1 \
    --trainer.max_epochs 200

# 3. Train with config file
# Train with a custom config file
echo -e "\nTraining with config file..."
anomalib train --config path/to/config.yaml

# 4. Export a trained model into OpenVINO
echo .e "\nExporting model into OpenVINO..."
anomalib export --model Patchcore \
        --ckpt_path /path/to/model.ckpt \
        --export_type OPENVINO
```

## Training

### Training with API

API CLI

```python
# 1. Import required modules
from anomalib.data import MVTecAD
from anomalib.deploy import ExportType
from anomalib.engine import Engine
from anomalib.models import Patchcore

# 2. Create a dataset
# MVTecAD is a popular dataset for anomaly detection
datamodule = MVTecAD(
    root="./datasets/MVTecAD",  # Path to download/store the dataset
    category="bottle",  # MVTec category to use
    train_batch_size=32,  # Number of images per training batch
    eval_batch_size=32,  # Number of images per validation/test batch
)

# 3. Initialize the model
# Patchcore is a good choice for beginners
model = Patchcore(
    num_neighbors=6,  # Override default model settings
)

# 4. Create the training engine
engine = Engine(
    max_epochs=1,  # Override default trainer settings
)

# 5. Train the model
# This produces a lightning model (.ckpt)
engine.fit(datamodule=datamodule, model=model)

# 6. Test the model performance
test_results = engine.test(datamodule=datamodule, model=model)

# 7. Export the model
# Different formats are available: Torch, OpenVINO, ONNX
engine.export(
    model=model,
    export_type=ExportType.OPENVINO,
)
```

Dataset 정의

Model 정의

Engine 정의

실제 학습

모델 테스트

모델 배포

## Inference

### Inference with API

API | CLI

```python
# 1. Import required modules
from anomalib.data import PredictDataset
from anomalib.engine import Engine
from anomalib.models import Patchcore

# 2. Initialize the model and load weights
model = Patchcore()
engine = Engine()

# 3. Prepare test data
# You can use a single image or a folder of images
dataset = PredictDataset(
    path="path/to/test/images",
    image_size=(256, 256),
)

# 4. Get predictions
predictions = engine.predict(
    model=model,
    dataset=dataset,
    ckpt_path="path/to/model.ckpt",
)

# 5. Access the results
if predictions is not None:
    for prediction in predictions:
        image_path = prediction.image_path
        anomaly_map = prediction.anomaly_map  # Pixel-level anomaly heatmap
        pred_label = prediction.pred_label  # Image-level label (0: normal, 1: anomalous)
        pred_score = prediction.pred_score  # Image-level anomaly score
```

추론할 데이터 정의
(폴더 or 이미지)

추론 시작
(학습된 모델 불러오기)

추론 결과 확인

## Visualization

### Direct Visualization

```python
from anomalib.visualization import visualize_anomaly_map
import torch

# Create sample anomaly map
anomaly_map = torch.rand(256, 256)

# Visualize with default settings
vis = visualize_anomaly_map(anomaly_map)

# Customize visualization
vis = visualize_anomaly_map(
    anomaly_map,
    colormap=True,      # Apply colormap
    normalize=True      # Normalize values to [0, 255]
)
```

## Visualization

- Visualization Callback (Validation / Test)

```python
from anomalib.visualization import ImageVisualizer

# Custom visualization settings
visualizer = ImageVisualizer(
    fields_config={
        "image": {},  # Default image display
        "anomaly_map": {
            "colormap": True,
            "normalize": True
        },
        "pred_mask": {
            "mode": "contour",
            "color": (255, 0, 0),
            "alpha": 0.7
        },
        "gt_mask": {
            "mode": "contour",
            "color": (0, 255, 0),
            "alpha": 0.7
        }
    }
)

# Pass visualizer to the model
model = Patchcore(visualizer=visualizer)
```

# Anomalib Customization



## Custom Dataset

- Folder datamodule 사용
- https://anomalib.readthedocs.io/en/v2.0.0/markdown/guides/how_to/data/custom_data.html

```python
# Import the datamodule
from anomalib.data import Folder

# Create the datamodule
datamodule = Folder(
    name="hazelnut_toy",
    root="datasets/hazelnut_toy",
    normal_dir="good",
    abnormal_dir="crack",
    mask_dir="mask/crack",
    normal_split_ratio=0.2,
)

# Setup the datamodule
datamodule.setup()
```

## VAD AI Project Process

### ➢ 본 강의에서의 실습 프로젝트 진행 범위

| 단계 | 핵심 질문 | 주요 결과물 |
|---|---|---|
| 1. 문제·요구사항 정의 | 어떤 결함을 왜 검출해야 하는가? | 결함 정의, 성공 기준 |
| 2. 데이터 정의 | 어떤 데이터로 개발·검증할 것인가? | 데이터 구성과 분할 |
| 3. 입력 설계 | 검사 정보를 어떤 영역·크기·형태로 전달할 것인가? | 기본 입력안·전처리 후보 |
| 4. 방법 설계 | 입력·학습 신호·자원 조건에 맞는 모델은 무엇인가? | 후보 모델·조합 비교 |
| 5. 운영 규칙 설계 | 모델 점수를 어떻게 최종 판정으로 바꿀 것인가? | 임계값·면적·재검사 규칙 |
| 6. 최종 시스템 설계 | 모델·입력·판정·저장을 어떻게 통합할 것인가? | 최종 설정과 파이프라인 |
| 7. 성능 분석·적용 판단 | 요구사항을 충족하며, 어디서 실패하는가? | 최종 지표·실패 사례·적용 조건 |

### ➢ 실무 적용을 위한 추가 프로세스

| 추가 단계 | 핵심 내용 |
|---|---|
| 8. 배포 및 인수시험 | 생산라인 연결, 처리속도, 인터페이스, 예외 처리 |
| 9. 운영 및 모니터링 | 성능 저하, 데이터 변화, 장비 변화 감시 |
| 10. 재검증·변경관리 | 모델·임계값·검사 규격 변경 시 재평가 |

## VAD AI Project Process




**입력안을 먼저 수립하고, 입력·모델 조합을 반복 검증함**
**모든 조건과 운영 규칙을 고정한 뒤 최종 평가를 수행함**

**반복 설계**

작은 결함 미검출 → 입력 해상도/방법 재검토
배경 과검출 → ROI/데이터 재검토
재검사율 과다 → 운영 규칙 재조정

**10주차: 제공된 MVTec AD 사례로 전 과정을 경험**
**→ 13~15주차: VAD 선택 시 자신의 업무·데이터로 동일한 과정을 수행**

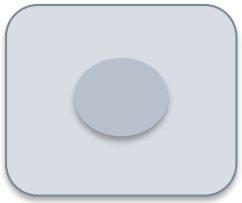
## 1. 문제·요구사항 정의

Q. 어떤 결함을 왜 검출해야 하며, 어느 수준까지 자동화할 것인가?

| 주요 결정 | 검사 대상 | 정상·불량 기준 | 성공·허용 기준 |
|---|---|---|---|
| Anomaly | 정상 데이터와 다른 패턴 | Defect | 품질 규격을 위반한 이상 |

**모델은 "다름"을 찾고, 최종 불량 여부는 품질 규격과 운영 규칙이 결정함**

### 정상



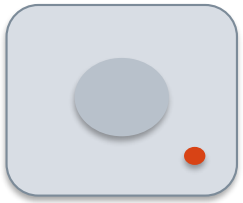
허용된
품질·외관 범위

### 정상 변동



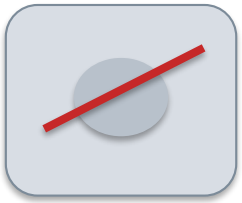

정상 범위 내
색상·반사 차이

### 허용 이상



정상과 다르지만
사용가능한 미세흠집

### 불량



규격을 초과한
스크래치·균열·오염

## 1. 문제·요구사항 정의

Q. 어떤 결함을 왜 검출해야 하며, 어느 수준까지 자동화할 것인가?

| 기술적 성공 기준 | | 운영적 성공 기준 | |
|---|---|---|---|
| 불량 검출률 | 95% 이상 | 최소 검출 결함 | 규격에서 정한 면적 이상 |
| 정상품 과검출률 | 5% 이하 | 재검사율 | 전체의 10% 이하 |
| 위치 검출 | 결함 위치 구분 가능 | 적용 범위 | 지정 제품·촬영조건 |
| 추론시간 | 100 ms/image 이하 | 자동 판단 범위 | 명확한 정상·불량 |

※ 위 수치는 강의용 예시이며, 실제 프로젝트에서는 현장 품질 규격과 생산 takt time을 근거로 설정함

# 1. 문제·요구사항 정의 (MVTec AD 실습 예시)

Q. 어떤 결함을 왜 검출해야 하며, 어느 수준까지 자동화할 것인가?

**실제 예시 사진 | MVTec AD bottle 이미지**



**정상 예시**

good



**결함 예시**

병 가장자리의 파손



**결함 확대**

빨간 윤곽은 정답 설명

**문제 정의: 정상 bottle 영상으로 정상 기준을 구성하고,**
**새 검사 영상의 이상 유무와 위치를 판단하여 통과·보류에 활용함**

# 1. 문제·요구사항 정의 (MVTec AD 실습 예시)

Q. 어떤 결함을 왜 검출해야 하며, 어느 수준까지 자동화할 것인가?

| 항목 | 이번 사례의 정의 | 성능 평가 시 확인할 내용 |
|---|---|---|
| 검사 단위 | bottle 영상 1장 = 검사 1건 | 이미지 단위 정상/불량 판정 |
| 불량 범위 | broken_large / broken_small / contamination | 작은 파손도 불량으로 평가 |
| 최우선 위험 | 불량을 정상으로 통과시키는 미검출 | 불량 Recall ≥ 95% |
| 재검사 부담 | 정상품의 불필요한 경보를 제한 | 정상품 과검출률 ≤ 5% |
| 처리시간 | 동일 PC · batch=1 · 전처리+추론+후처리 | ≤ 100 ms [가정] |
| 적용 범위 | MVTec bottle의 관찰 가능한 영상 범위 | 실제 라인·다른 품종은 별도 검증 |

## 2. 데이터 정의

Q. 어떤 조건에서 수집된 데이터를 어떤 목적으로 사용할 것인가?

**주요 결정** ▶ **데이터·라벨 현황** | **촬영·공정 조건** | **분할·품질 기준**

| 데이터 정의 범위 | 실제 기업 데이터에서 함께 확인할 촬영 조건 |
|---|---|
| 이미지 | 1 카메라·렌즈·해상도 |
| 정상·불량 라벨 | 2 조명 방향·밝기·반사 |
| 결함 위치 마스크 | 3 촬영 거리·제품 위치·지그 |
| 제품·공정 조건 | 4 트리거 시점·생산 속도 |
| 촬영 조건 | 5 품종·Lot·원자재·운전 조건 |

MVTec AD에서는 촬영조건이 정리되어 있지만,
**실제 프로젝트에서는 영상 획득 시스템도 데이터 정의에 포함함**

## 2. 데이터 정의

Q. 어떤 조건에서 수집된 데이터를 어떤 목적으로 사용할 것인가?

| 정상 학습 데이터 (Training Data) | | 검증 데이터 (Validation Data) | | 최종 평가 데이터 (Test Data) | |
|---|---|---|---|---|---|
| 사용 목적 | 정상 기준 구성 | 사용 목적 | 방법·입력·임계값 선택 | 사용 목적 | 고정된 시스템의 성능 확인 |
| 금지 | 최종 성능 보고에 사용하지 않음 | 금지 | 최종 성능으로 보고하지 않음 | 금지 | ROI·임계값 등 실험 조건 재조정 금지 |

검증 데이터로 설계안을 선택 → 조건 고정 → 최종 평가

**최종 평가 결과를 보고 설정을 변경하면 해당 데이터는 더 이상 독립적인 최종 평가 데이터가 아님!**

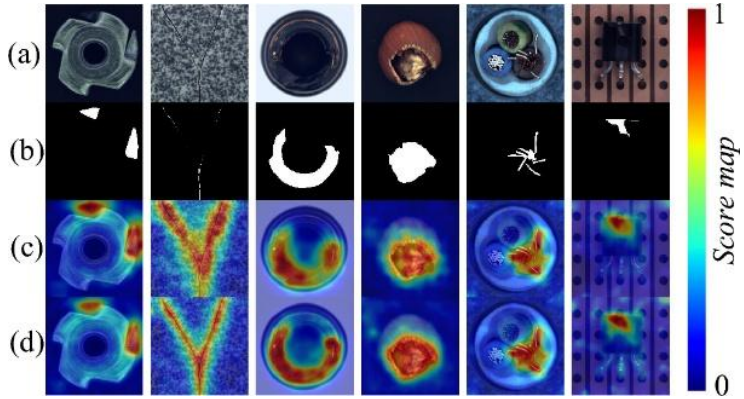
## 2. 데이터 정의

Q. 어떤 조건에서 수집된 데이터를 어떤 목적으로 사용할 것인가?

**확인 체크리스트**

- ✓ **정상·이상 이미지 수**
- ✓ **결함 유형별 수량**
- ✓ 이미지 크기·채널·파일 형식
- ✓ **결함 마스크 존재 여부**
- ✓ **분할 간 파일 중복**
- ✓ 촬영·제품 변동
- ✓ 결함 크기 분포

| Split | Normal | Abnormal | Mask | 용도 |
|---|---|---|---|---|
| Train | — | 0 | 없음 | 정상 기준 구성 |
| Validation | — | — | 있음 | 설계안 선택 |
| Final Test | — | — | 있음 | 최종 평가 |




원본 이미지 · 정답 마스크 · 이상 지도 사례

## 2. 데이터 정의 (MVTec AD 실습 예시)

Q. 어떤 조건에서 수집된 데이터를 어떤 목적으로 사용할 것인가?

**MVTec AD: 15개 범주 · 총 5,354장 | 본 실습: bottle, 총 292장**

| 항목 | bottle 원본 데이터 |
|---|---|
| 영상 크기 | 900 × 900 픽셀 · RGB 3채널 |
| 파일 형식 | 영상: PNG / 결함 마스크: 흑백 PNG |
| 결함 유형 | broken_large · broken_small<br>contamination (총 3종) |

| 분할 | 정상 | 이상 | 합계 |
|---|---|---|---|
| 공식 Train | 209 | 0 | 209 |
| 공식 Test | 20 | 63 | 83 |

**원본 자료의 이미지 예시**



정상 영상



불량 영상



마스크의 흰 영역:
결함 위치 정답

모델 입력이 아님

## 2. 데이터 정의 (MVTec AD 실습 예시)

Q. 어떤 조건에서 수집된 데이터를 어떤 목적으로 사용할 것인가?

**재분할 예시 | Train은 유지하고 원본 Test 83장을 Validation / Final Test로 구분함**

| Split | 정상 | 이상 | 마스크 | 용도 |
|---|---|---|---|---|
| Train | 209 | 0 | 없음 | 정상 기준 구성 |
| Validation | 10 | 31 | 있음 | 입력·방법·운영 규칙 선택 |
| Final Test | 10 | 32 | 있음 | 조건 고정 후 최종 평가 |



불량 영상 예시



정답 마스크 예시

**정답 마스크 ≠ 모델 이상 지도**

**마스크는 위치 평가를 위한 정답임**

**학습이나 추론 입력, 결함 점수 생성에 사용하지 않음**

**검증용 마스크로만 픽셀 임계값을 선택함**

# 3. 입력 설계

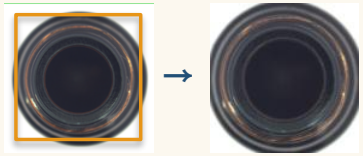
Q. 검사에 필요한 정보는 유지하면서 불필요한 변동은 어떻게 제거할 것인가?

**주요 결정** | **ROI·검사 영역** | **해상도·정렬** | **정규화·전처리**

원본 이미지 → ROI → 정렬 → Resize / Crop → 정규화 → 모델 입력

### ① ROI · 검사 영역

검사할 제품 영역을 지정함
배경을 줄이되 결함은 보존함



전체 → 제품 ROI

### ② 정렬 · 위치 대응

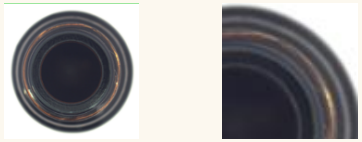
위치·각도 편차를 줄임
구조의 대응 위치를 맞춤



위치 편차 → 기준 위치

### ③ Resize / Crop

Resize: 영상 크기를 변경함
Crop: 일부 영역을 선택함



전체 축소 | 일부 선택

### ④ 정규화 · 수치 변환

픽셀의 수치 범위를 맞춤
모델별 기준을 적용함



128 / 255
≈ 0.502

채널별 $z = (x - \mu) / \sigma$

## 4. 방법 설계

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

| 순서 | 핵심 질문 | 결정 사항 |
|---|---|---|
| ① 학습 신호 | 모델 학습에 실제 정상·이상 정답을 사용할 수 있는가?<br>정상만 사용할 것인가? 합성 이상을 만들 수 있는가? | 지도·비지도·자기지도 접근 |
| ② 예측 수준 | 제품의 이상 유무만 필요한가,<br>이상 위치와 영역까지 필요한가? | Image·Patch·Pixel 수준의 출력 조건 |
| ③ 데이터와 메커니즘의 적합성 | 위치별 정상 변동을 요약하는 것이 적합한가?,<br>다양한 정상 사례와 비교하는 것이 적합한가?,<br>복원과 판별을 학습할 수 있는가? | 분포·거리·복원 기반 후보 |
| ④ 학습·실행 제약 | 신경망을 새로 학습할 수 있는가?<br>정상 특징의 저장·검색 비용을 감당할 수 있는가? | 후보 유지·제외·우선순위 |
| ⑤ 후보군 확정 | 각 후보를 포함하는 이유와 실험으로 확인해야 할 위험은 무엇인가? | 비교 대상과 검증 항목 |

# 4. 방법 설계

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

① 학습 신호

| 질문에 대한 답변 | 연결되는 접근 | 검토할 후보 |
|---|---|---|
| **실제 정상·이상 영상과 신뢰할 수 있는 라벨이 충분함** | 지도학습 Supervised | 이미지 분류 모델<br>위치 정답이 있으면 분할 모델 |
| **실제 학습 데이터는 정상뿐임<br>실제 이상 라벨은 학습에 사용하지 않음** | 비지도학습 Unsupervised | PaDiM · PatchCore<br>정상 복원 기반 모델 |
| **정상 영상에 합성 이상을 넣고<br>변형 위치를 학습 정답으로 만들 수 있음** | 자기지도 접근 Self-supervised | DRAEM<br>합성 이상 기반 복원·판별 학습 |

**정상 데이터만 수집했어도 합성 이상을 만들 수 있음 → 비지도·자기지도 후보가 함께 남을 수 있음.**

# 4. 방법 설계

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

② 예측 수준

| 업무 요구에 대한 답변 | 필요한 출력 | 후보를 남기는 기준 |
|---|---|---|
| **제품의 통과·불량만 구분하면 됨** | Image-level<br>이미지 점수·판정 | 이미지 단위 출력이 가능한 접근을 후보로 둠 |
| **작업자가 이상 위치를 확인해야 함** | Patch-level<br>위치별 지도 | 위치 출력을 제공하지 않는 방법은 보완하거나 제외함 |
| **결함 영역·면적을 계산해 판정해야 함** | Pixel-level<br>지도·마스크 | 영역을 산출할 수 있는 후보를 남기고 경계·작은 결함 성능을 검증함 |

패치 기반 방법도 픽셀 크기의 지도를 출력할 수 있음. 출력 해상도가 실제 위치 정확도를 보장하지는 않음.

# 4. 방법 설계

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

**③ 데이터-메커니즘 적합성**

| 설계 질문 → "예"일 때 | 접근·모델 후보 | 검토 이유 | 확인할 위험 |
|---|---|---|---|
| **제품 위치·구조가 비교적 일정하여 같은 위치의 정상 변동을 비교하는 것이 의미 있는가?** | Density-based<br>PaDiM | 위치별 정상 특징 분포를 구성할 수 있음 | 위치 변화<br>분포 가정<br>정상 표본 범위 |
| **여러 정상 패턴을 대표 사례로 보관하고 유사한 정상 패치가 있는지 비교할 수 있는가?** | Distance-based<br>PatchCore | 대표 패치의 특징 유사도를 활용할 수 있음 | 패턴 누락<br>저장량<br>검색시간 |
| **합성 이상을 만들고, 정상 복원과 이상 영역 판별을 함께 학습할 수 있는가?** | Reconstruction<br>+ Discrimination<br>DRAEM | 합성 위치 정답으로 판별을 학습할 수 있음 | 실제 결함으로의 일반화<br>학습 비용 |

# 4. 방법 설계

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

④ 학습/실행 제약

| 질문에 대한 답변 | 후보군에 반영 | 함께 확인할 조건 |
|---|---|---|
| **사전학습 특징은 사용 가능하나 신경망 재학습 시간을 줄여야 함** | PaDiM · PatchCore 우선 검토 | 정상 분포 추정·메모리뱅크 구성시간도 측정함 |
| **정상 특징의 저장·검색 비용이 제약됨** | PatchCore 설정· 적용 여부 검토 | coreset 축소의 성능 손실과 검색시간을 함께 확인함 |
| **위치별 고차원 통계량의 저장이 부담됨** | PaDiM 설정· 적용 여부 검토 | 특징 차원·공간 크기를 조정하고 자원과 위치 성능을 비교함 |
| **합성 이상 생성과 반복 학습에 필요한 시간·장비를 확보함** | DRAEM 추가 검토 | 학습 조건과 실제 결함에 대한 검증 결과를 기록함 |

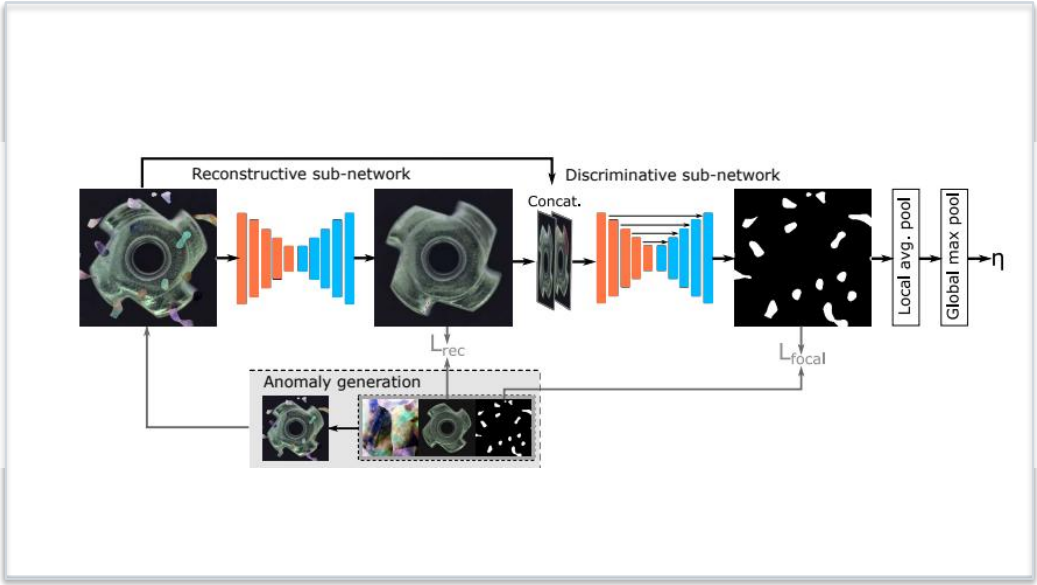
# 4. 방법 설계

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

**핵심 원리**

합성 이상이 포함된 입력을 정상처럼 복원하고, 입력·복원 결과를 이용해 이상 영역을 직접 판별함

| 설계 관점 | DRAEM |
|---|---|
| **학습 데이터** | 정상 + 합성 이상 |
| **정상성 표현** | 정상 복원 패턴 |
| **이상 점수** | 판별 네트워크 출력 |
| **장점** | 픽셀 수준 이상 위치 학습 |
| **위험** | 합성 이상 품질·학습시간 |




**픽셀 수준 이상 위치 학습이 가능하지만,
데이터 셋 별 신경망 학습이 요구됨**

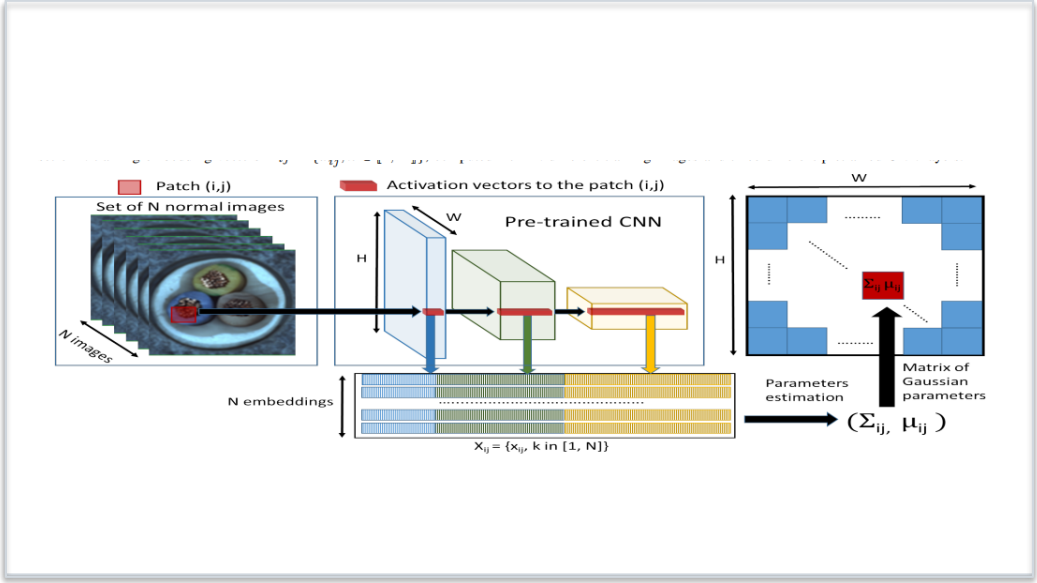
# 4. 방법 설계

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

**핵심 원리**

사전학습 CNN의 다층 패치 특징을 추출하고, 각 공간 위치의 정상 특징을 가우시안 분포로 모델링함

| 설계 관점 | PaDiM |
|---|---|
| 학습 데이터 | 정상 이미지 |
| 정상성 표현 | 위치별 평균·공분산 |
| 이상 점수 | Mahalanobis distance |
| 장점 | 별도 CNN 학습 부담이 작음 |
| 위험 | 정렬·위치 변화에 민감 가능 |




**사전학습 특징을 사용하므로 재학습 부담이 작지만, 각 패치 단위별 분포를 모델링하므로 정렬 위치 변화 민감**

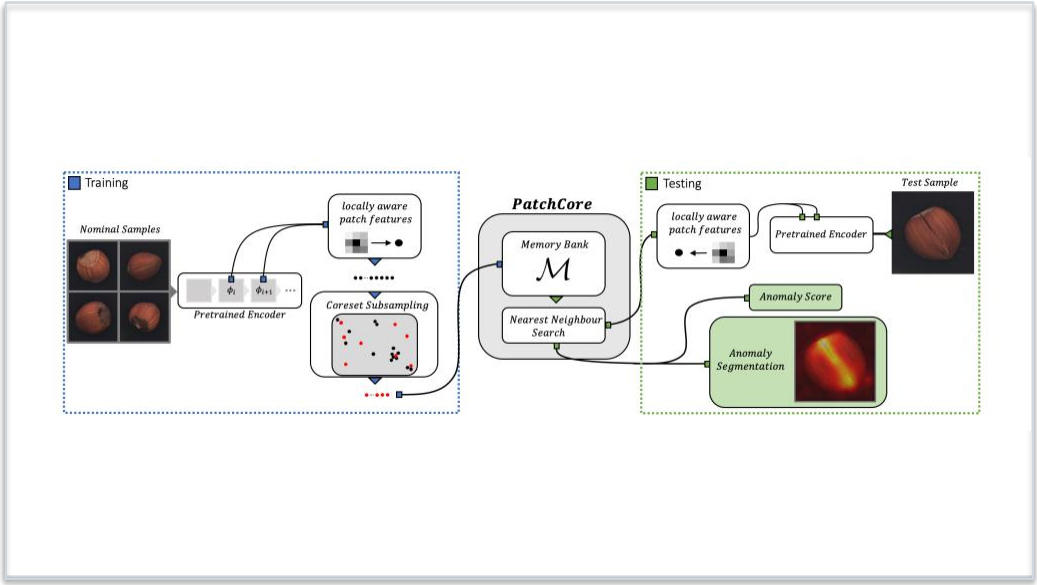
# 4. 방법 설계

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

**핵심 원리**

정상 패치 특징을 수집한 뒤 대표 특징만 Coreset으로 저장하고, 검사 특징과 가장 가까운 정상 특징의 거리를 계산함

| 설계 관점 | PatchCore |
|---|---|
| 학습 데이터 | 정상 이미지 |
| 정상성 표현 | 대표 패치 특징 집합 |
| 이상 점수 | Nearest-neighbor distance |
| 장점 | 강력한 사전학습 특징 활용 |
| 위험 | 메모리 크기·검색시간 |




**사전학습 특징을 사용하므로 재학습 부담이 작지만, 메모리 뱅크를 위한 저장 공간이 요구되며, 메모리 뱅크가 클수록 검색 시간이 지연될 수 있음**

## 4. 방법 설계 (MVTec AD 실습 예시)

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

**데이터 사실**



정상 학습 영상 사용
이상 영상·마스크는
검증·평가에만 사용함

| | |
|---|---|
| **학습 신호** | 실제 이상 정답은 학습에 사용하지 않음<br>→ **비지도학습/자기지도학습 기반 접근** |
| **필요 출력** | 파손·오염의 위치와 영역까지 필요함<br>→ **Patch-level, Pixel-level 후보** |
| **특징·자원** | 사전학습 CNN과 정상 기준 구성이 가능함<br>→ **PaDiM · PatchCore** |
| **합성·재학습** | 합성 이상 생성과 반복 학습이 가능함<br>→ **DRAEM 추가 검토** |

**학습 조건에 맞는 후보를 남긴 결과: PaDiM · PatchCore + (합성·재학습 가능 시) DRAEM**

## 4. 방법 설계 (MVTec AD 실습 예시)

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

**각 후보를 선택한 이유를 질문의 답과 연결하고, 아직 확인하지 못한 점을 명시함**

| 후보 | 포함 근거 | 예상 이점 | 반드시 검증할 위험 |
|---|---|---|---|
| **M1 PaDiM** | 정상 중심 학습 가능함<br>위치 지도가 필요함<br>검사 대상 구조가 일정함 | 사전학습 특징으로<br>정상 분포를 구성함 | 위치 변동<br>분포 가정·표본 범위<br>통계량 저장 비용 |
| **M2 PatchCore** | 정상 패치 사례를 저장 가능<br>검색 비용을 감안할 수 있음<br>위치 지도가 필요함 | 대표 정상 특징과의<br>유사도를 활용함 | 정상 패턴 누락<br>메모리뱅크 크기<br>검색시간 |
| **M3 DRAEM** | 정상에서 합성 이상을 만듦<br>학습 자원을 확보함<br>위치 지도가 필요함 | 합성 위치 정답으로<br>복원·판별을 학습함 | 실제 결함 일반화<br>합성 이상 적합성<br>학습 비용 |

# 4. 방법 설계

Q. 어떤 학습 신호와 출력이 필요하며, 데이터·자원 조건에 맞는 모델 후보는 무엇인가?

## 모델 비교 실험 가이드

### 공통으로 고정할 조건

- ✓ 동일한 학습·검증·최종 평가 이미지
- ✓ 모델 비교 시 동일 해상도·ROI
- ✓ 동일한 실행 장치·시간 측정 범위
- ✓ 동일한 후처리·평가 지표
- ✓ 고정된 난수 seed와 파일 목록

### 모델별로 별도 기록할 조건

- i 백본·특징층·모델별 정규화
- i DRAEM 학습 epoch·합성 이상
- i PaDiM 특징 차원·분포 저장량
- i PatchCore coreset 비율·메모리
- i 학습/기준구성 시간·추론시간

# 4. 방법 설계 (MVTec AD 실습 예시)

Q. 입력안과 후보 모델 중 어떤 조합이 검사 요구사항을 충족하는가?

| 모델 | 불량 Recall | 정상 과검출률 | 위치 성능 | 추론시간 | 메모리 | 대표 실패 |
|---|---|---|---|---|---|---|
| DRAEM | — | — | — | — | — | — |
| PaDiM | — | — | — | — | — | — |
| PatchCore | — | — | — | — | — | — |

운영 규칙 설계에는
_____ 입력안·모델을 사용함

선택 이유:
_____

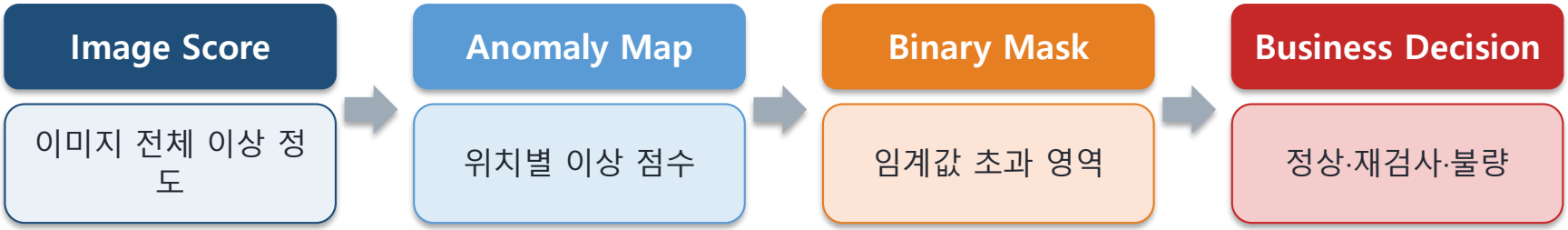
가장 큰 위험:
_____

추가 검증이 필요한 조건:
_____

## 5. 운영규칙 설계

Q. 이상 점수·이상 지도를 정상·재검사·불량 판정으로 어떻게 변환할 것인가?

**주요 결정** | **점수·픽셀 임계값** | **결함 면적·형태** | **재검사·예외 처리**




**모델이 제공하는 값**

- 이미지 이상 점수
- 패치/픽셀 이상 지도

**현장에서 추가로 결정할 것**

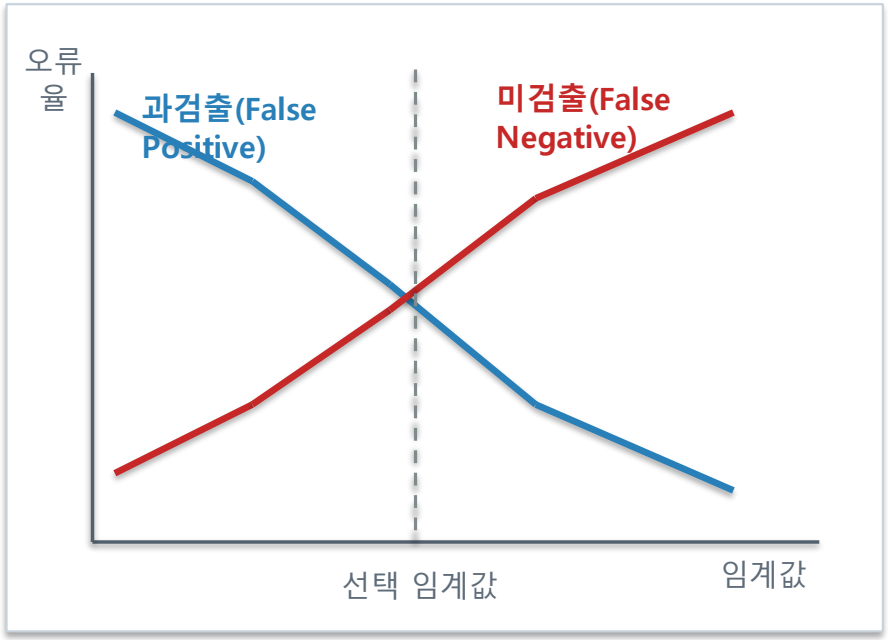
- 불량 허용 임계값
- 재검사 구간·작업자 확인
- 자동 통과·자동 제거 조건

**모델의 "이상"과 업무의 "불량" 사이를 연결하는 것이 운영 규칙 설계임**

## 5. 운영규칙 설계

Q. 이상 점수·이상 지도를 정상·재검사·불량 판정으로 어떻게 변환할 것인가?

**VAD 임계값과 종류**




**이미지 임계값 $\tau_i$**

제품 전체의 정상·불량 판정

**픽셀 임계값 $\tau_p$**

이상 지도에서 결함 후보 영역 생성

**운영 원칙**

미검출 비용이 큰 공정은 낮은 임계값과 재검사 절차를 함께 설계할 수 있음

**임계값은 최종 테스트가 아니라 검증 데이터와 업무 위험 기준으로 확정함**

# 5. 운영규칙 설계

Q. 이상 점수·이상 지도를 정상·재검사·불량 판정으로 어떻게 변환할 것인가?

**정상/재검사/불량 판정 임계값**

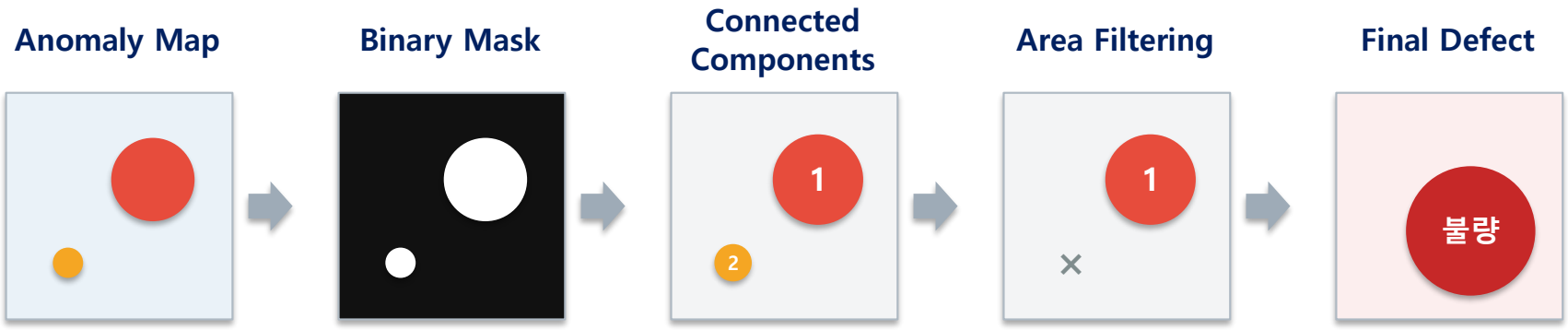
| | |
|---|---|
| 정상 | Score < τ_normal |
| 재검사 | τ_normal ≤ Score < τ_defect |
| 불량 | Score ≥ τ_defect |

| 규칙 | 판정 방식 | Recall | 과검출 | 재검사율 | 선택 |
|---|---|---|---|---|---|
| A | 이미지 점수 임계값 | — | — | - | |
| B | 점수 + 최소 결함 면적 | — | — | - | |
| C | 정상·재검사·불량 | — | — | — | |

# 5. 운영규칙 설계

Q. 이상 점수·이상 지도를 정상·재검사·불량 판정으로 어떻게 변환할 것인가?

## 이상 지도 임계값




| 수식 | 계산 의미 |
|---|---|
| $M(u,v) = 1[A(u,v) \geq \tau_p]$ | $\tau_p$ 이상: 1 / 미만: 0<br>결함 후보 마스크를 생성함 |
| $A_{max} = \max \text{Area}(c)$<br>$c \in \text{ConnectedComponents}(M)$ | 연결된 후보 영역 중<br>최대 픽셀 수를 계산함 |
| Defect = $(s_image \geq \tau_i)$<br>AND $(A_{max} \geq A_min)$ | 이미지 점수·최소 면적이<br>모두 기준 이상이면 불량임 |

**예시 계산 · 실측 결과 아님**

연결 영역 면적이 120, 8이면
$A_{max}$ = 120 픽셀임
A_min = 40이면 면적 조건을 통과함

**픽셀 면적은 입력 크기에 따라 달라짐**
**작은 실제 결함도 제거될 수 있음**

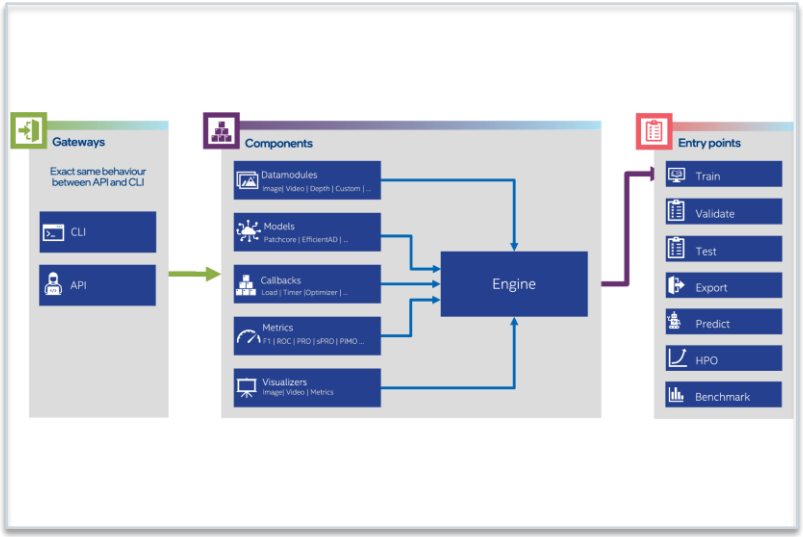
## 6. 시스템 설계

검사 이미지 → 입력 전처리 → VAD 모델 → 모델 출력 → 운영 규칙 → 최종 판정·저장

| 입력 전처리 | VAD 모델 | 모델 출력 | 운영 규칙 |
|---|---|---|---|
| ROI · Resize<br>Normalize | DRAEM /<br>PaDiM<br>PatchCore | Image Score<br>Anomaly Map | Threshold ·<br>Area<br>Review Zone |

### Anomalib 구성과 시스템 요소의 연결

| 시스템 요소 | Anomalib |
|---|---|
| 데이터 입력·전처리 | Datamodule |
| VAD 방법 | Model |
| 학습·검증·추론 | Engine |
| 성능지표 | Metrics |
| 이상 지도·결과 이미지 | Visualizer |



## 7. 성능 분석 및 평가

Q. 성능이 충분한가, 어디에서 실패하며, 어떤 조건에서 현장 적용이 가능한가?

**주요 결정** | 최종 성능 검증 | 실패 원인 분석 | 적용·추가 검증 판단

**설계 검증**

검증 데이터 → 모델·입력·규칙 비교 조건 수정·재검증

**최종 성능 검증**

모든 조건 고정 → 사용하지 않은 Final Test

**성능지표는 세 수준으로 구성함**

| 모델 수준 | 업무 수준 | 시스템 수준 |
|---|---|---|
| Image/Pixel AUROC | 불량 Recall · 과검출률 재검사율 · 크기별 Recall | 추론시간 · 메모리 처리 실패율 |

# 7. 성능 분석 및 평가

Q. 성능이 충분한가, 어디에서 실패하며, 어떤 조건에서 현장 적용이 가능한가?

**주요 평가 지표 ① 이미지 판정 · 불량을 양성(Positive)으로 정의함**

**TP: 불량→불량 FN: 불량→정상 FP: 정상→불량 TN: 정상→정상**

| 평가지표 | 계산식 | 의미·해석 |
|---|---|---|
| Recall / 검출률 ↑ | TP / (TP + FN) | 실제 불량 중 검출한 비율임 |
| 미검출률(FNR) ↓ | FN / (TP + FN) | 실제 불량을 정상으로 잘못 통과시킨 비율임 |
| 정상 과검출률(FPR) ↓ | FP / (FP + TN) | 정상품을 불량으로 잘못 판단한 비율임 |
| Precision / 정밀도 ↑ | TP / (TP + FP) | 불량 판정 중 실제 불량인 비율임 |
| F1-score ↑ | 2TP / (2TP + FP + FN) | 정밀도와 재현율의 조화평균임 |
| Accuracy / 정확도 ↑ | (TP + TN) / N | 전체 정답률임. 클래스 불균형에 주의함 |

N = TP + FN + FP + TN. 이 표는 재검사 없는 이진 판정 기준이며, 분모가 0인 경우 별도 표기함.

# 7. 성능 분석 및 평가

Q. 성능이 충분한가, 어디에서 실패하며, 어떤 조건에서 현장 적용이 가능한가?

**주요 평가 지표 ①  이미지 판정 · 불량을 양성(Positive)으로 정의함**

**TP: 불량→불량  FN: 불량→정상  FP: 정상→불량  TN: 정상→정상**

| | | Predicted Class: Positive | Predicted Class: Negative | |
|---|---|---|---|---|
| Actual Class | Positive | True Positive (TP) | False Negative (FN) **Type II Error** | **Sensitivity** $\frac{TP}{(TP+FN)}$ |
| | Negative | False Positive (FP) **Type I Error** | True Negative (TN) | **Specificity** $\frac{TN}{(TN+FP)}$ |
| | | **Precision** $\frac{TP}{(TP+FP)}$ | **Negative Predictive Value** $\frac{TN}{(TN+FN)}$ | **Accuracy** $\frac{TP+TN}{(TP+TN+FP+FN)}$ |

# 7. 성능 분석 및 평가

Q. 성능이 충분한가, 어디에서 실패하며, 어떤 조건에서 현장 적용이 가능한가?

## 주요 평가 지표 ②  이상 점수 · 결함 위치 품질

| 평가지표 | 계산식 / 계산 대상 | 의미·해석 |
|---|---|---|
| Image / Pixel AUROC ↑ | ∫[0,1] TPR(f) df  (f = FPR)<br>이미지 점수 / 전체 픽셀 점수로 계산 | 임계값 전반의 정상·이상 구분 능력임<br>선택한 운영점의 오류율과는 다름 |
| IoU ↑ | \|P ∩ G\| / \|P ∪ G\| | 예측 결함 영역과 정답의 겹침임<br>검증에서 정한 픽셀 임계값을 사용함 |

P: 예측 결함 픽셀 집합 · G: 정답 결함 집합

# 7. 성능 분석 및 평가

Q. 성능이 충분한가, 어디에서 실패하며, 어떤 조건에서 현장 적용이 가능한가?

**주요 평가 지표 ②  이상 점수 · 결함 위치 품질**

## 임계값을 움직이면, 같은 점수도 판정이 달라짐

가상 점수 6개 · 점수가 클수록 이상하다고 보는 경우 · 점수는 불량 확률이 아님

| 정상 | 정상 | 불량 | 정상 | 불량 | 불량 |
|---|---|---|---|---|---|
| 0.1 | 0.3 | 0.4 | 0.6 | 0.8 | 0.9 |

이상 점수가 커지는 방향 →

| 임계값 τ (점수 ≥ τ → 불량) | 잡은 불량 / 실제 불량 (Recall, TPR) | 잘못 잡은 정상 / 실제 정상 (FPR) |
|---|---|---|
| 0.70 | 2 / 3 = 66.7% | 0 / 3 = 0% |
| 0.50 | 2 / 3 = 66.7% | 1 / 3 = 33.3% |
| 0.35 | 3 / 3 = 100% | 1 / 3 = 33.3% |

**고정 임계값에서의 Recall·FPR은 "운영점", 여러 임계값을 훑는 평가는 "곡선"임**

## 7. 성능 분석 및 평가

Q. 성능이 충분한가, 어디에서 실패하며, 어떤 조건에서 현장 적용이 가능한가?

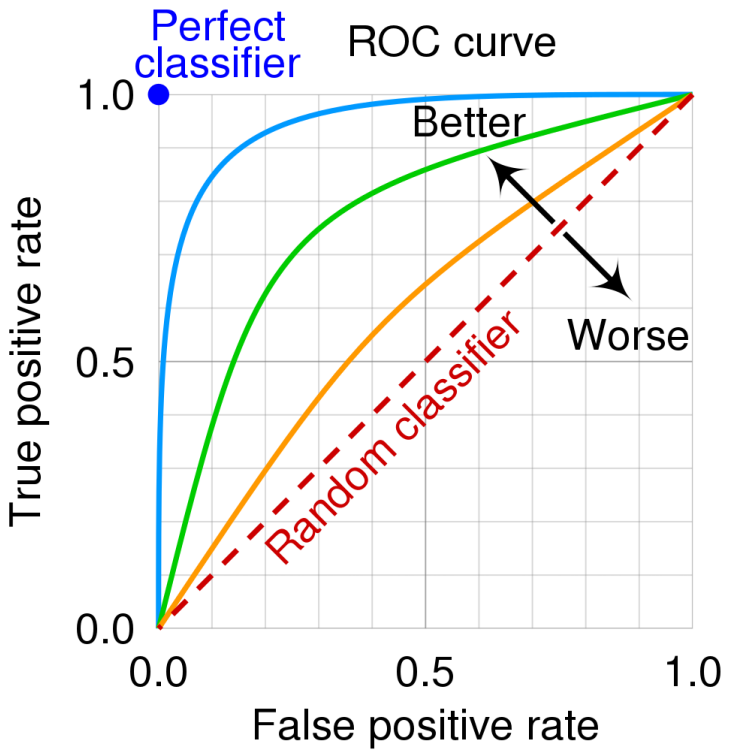
**주요 평가 지표 ②  이상 점수 · 결함 위치 품질**

**AUROC (Area Under ROC Curve),**
**AUC-ROC (Area Under Curve of ROC Curve)**

> ROC Curve의 아래쪽 넓이

* ROC Curve : 분류 모델의 임계값(Threshold, 보통 0~1)을 바꾸어가며, FPR(False Positive Rate, 위양성률)을 X축에, TPR(True Positive Rate, 민감도/재현율)을 Y축에 놓고 그린 곡선

> 임계값에 무관한 모델의 성능 확인



## 7. 성능 분석 및 평가

Q. 성능이 충분한가, 어디에서 실패하며, 어떤 조건에서 현장 적용이 가능한가?

**주요 평가 지표 ②  이상 점수 · 결함 위치 품질**

### Image AUROC vs Pixel AUROC

**Image-level: 병 사진 1장씩**

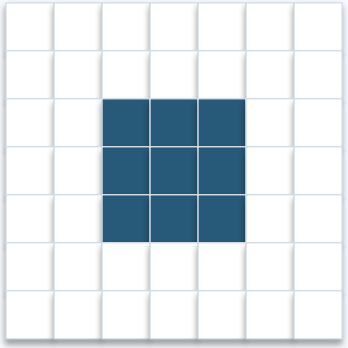


정상 이미지



불량 이미지

**Pixel-level: 사진 속 픽셀씩**



# 7. 성능 분석 및 평가

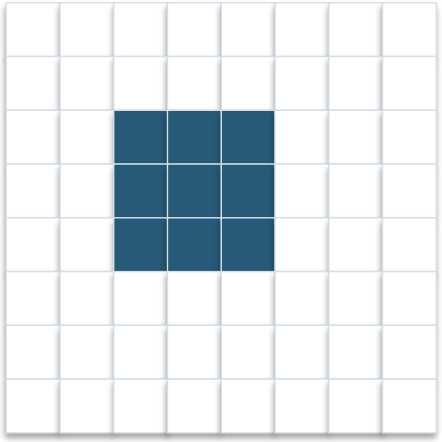
Q. 성능이 충분한가, 어디에서 실패하며, 어떤 조건에서 현장 적용이 가능한가?

**주요 평가 지표 ②  이상 점수 · 결함 위치 품질**

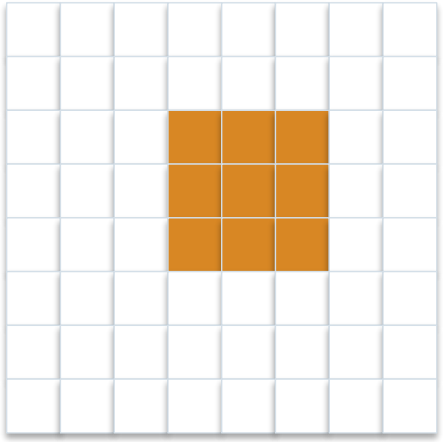
## IoU (Intersection over Union)

설명용 8×8 격자 · 동일한 좌표와 해상도에서 비교함

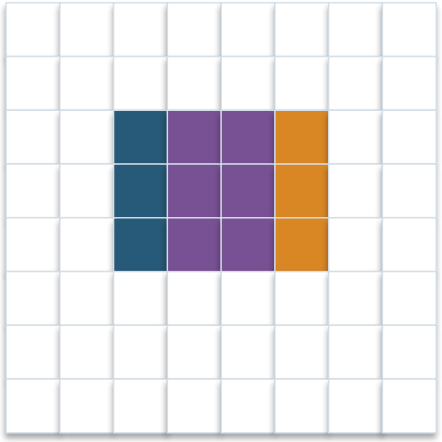
**정답 G: 9픽셀**



**예측 P: 9픽셀**



**겹친 부분: 6픽셀**



놓친 정답 3

맞게 겹침 6

잘못 칠함 3

# 7. 성능 분석 및 평가

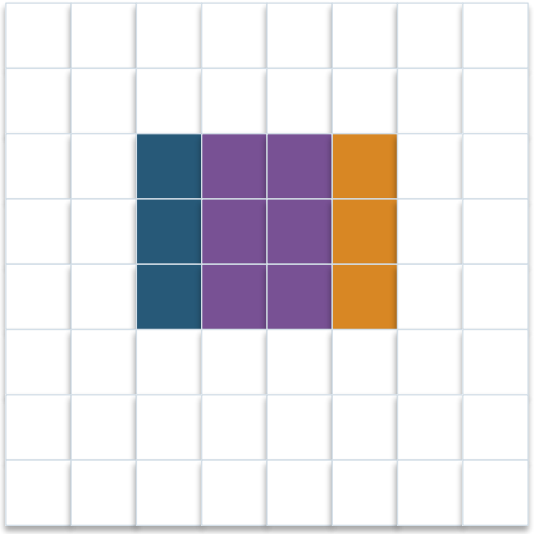
Q. 성능이 충분한가, 어디에서 실패하며, 어떤 조건에서 현장 적용이 가능한가?

**주요 평가 지표 ②  이상 점수 · 결함 위치 품질**

## IoU (Intersection over Union)

설명용 8×8 격자 · 동일한 좌표와 해상도에서 비교함

Intersection over Union = 교집합 ÷ 합집합



**파랑 + 보라 + 주황 = 합집합 12**

**IoU ↑**

**6 / 12 = 50%**

겹친 픽셀 6개
÷ (정답 9 + 예측 9 – 겹침 6)

# 7. 성능 분석 및 평가

Q. 성능이 충분한가, 어디에서 실패하며, 어떤 조건에서 현장 적용이 가능한가?

**실패사례 분석**

| 실패 유형 | 가능한 원인 | 다시 검토할 단계 |
|---|---|---|
| **배경을 결함으로 판단** | 불필요한 배경 포함 | 입력 설계 |
| **정상 구조를 결함으로 판단** | 정상 변동 데이터 부족 | 데이터 정의 |
| **작은 결함 미검출** | 해상도·특징 크기 부족 | 입력·방법 설계 |
| **새로운 반사에 반응** | 조명·촬영 변화 | 데이터·획득 조건 |
| **ROI 밖 결함 미검출** | ROI가 너무 좁음 | 입력 설계 |
| **작은 노이즈 과검출** | 면적 규칙 부재 | 운영 규칙 |

# 차후 수업에 관하여...

INDUSTRIAL AI RESEARCH CENTER

## ■ 차후 수업 진행 안내

| 주차 | 수업내용 | 수업방식 |
|---|---|---|
| 1 | Lecture Introduction | 대면수업 |
| 2 | Introduction to Industrial AI | 대면수업 |
| 3 | Basic of Neural Network | 대면수업 |
| 4 | Learning Strategies: Supervised Learning | 대면수업 |
| 5 | Learning Strategies: Unsupervised Learning | 대면수업 |
| 6 | 글로벌 제조 트렌드 특강 | 비대면수업 |
| 7 | Model Architecture: Convolutional Neural Networks | 대면수업 |
| 8 | Model Architecture: Sequence Models | 대면수업 |
| 9 | Midterm exam | 대면수업 |
| 10 | Visual Anomaly Detection | 대면수업 |
| 11 | Temporal Anomaly Detection | 대면수업 |
| 12 | Predictive Maintenance with Time Series | 대면수업 |
| 13 | Final Project: Proposal | 대면수업 |
| 14 | Final Project: Discussion | 대면수업 |
| 15 | Final Project: Presentation | 대면수업 |

# 감사합니다

# Q&A